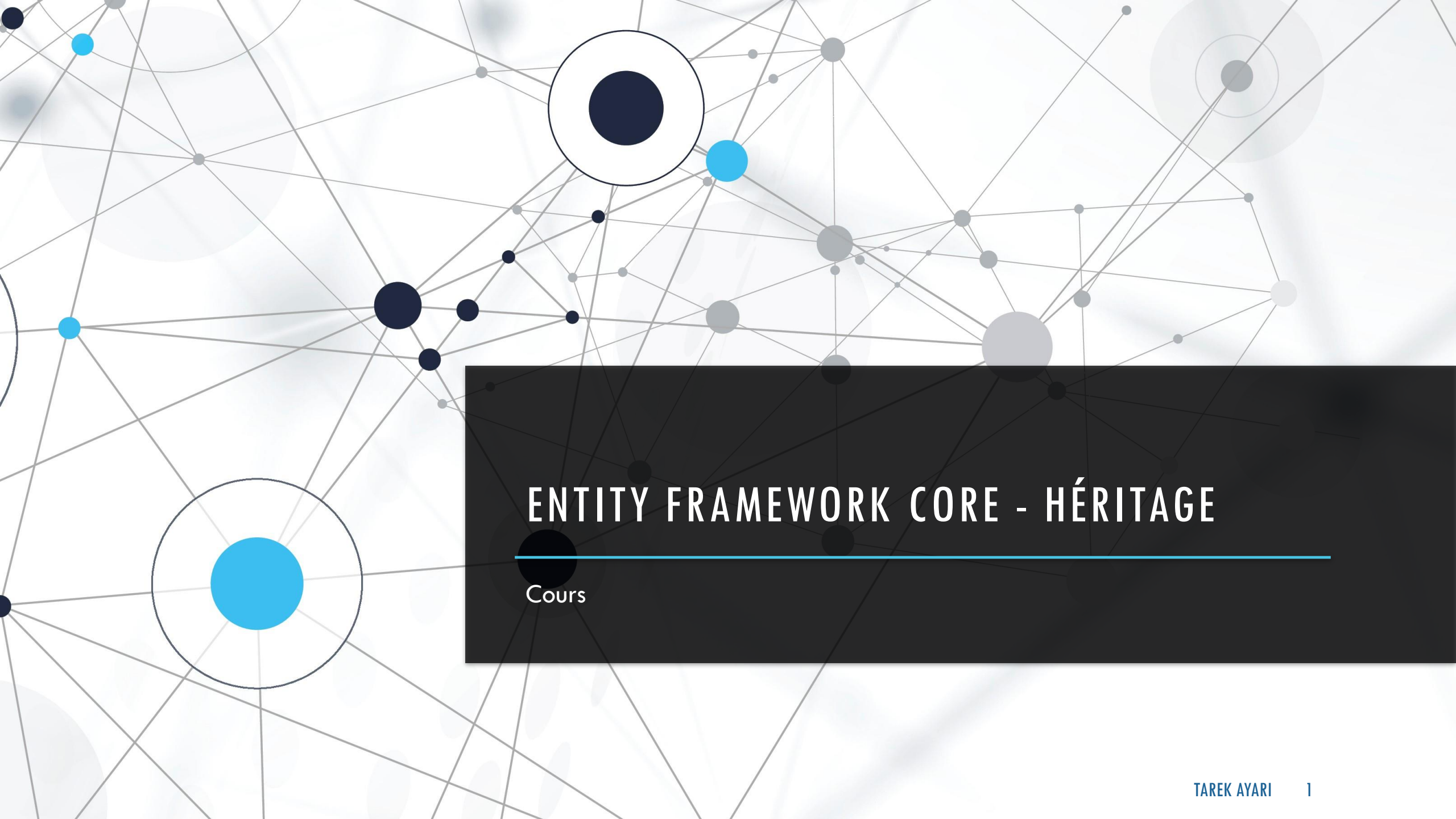


ENTITY FRAMEWORK CORE - HÉRITAGE

Cours

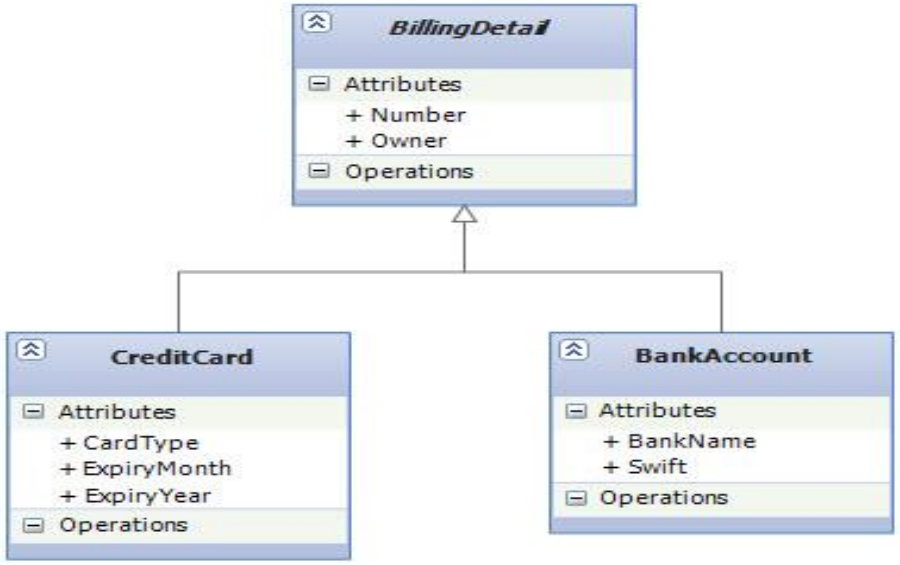
INHERITANCE IN ENTITY FRAMEWORK CODE-FIRST

Lorsque nous utilisons "Entity Framework Code First" pour définir notre modèle , il existe différentes façons pour définir l'héritage dans notre base de données:

- **Table Per Hierarchy (TPH):** Il n'y a qu'une seule table et un discriminant pour savoir sur quel type on se trouve.
- **Table Per Type (TPT):** Il y a une table partagée qui possède les propriétés de la classe de base et une table pour chaque classe héritée avec les propriétés qui lui sont spécifiques.

TABLE PER HIERARCHY (TPH)

- Dans ce modèle il y a une seule table qui représente toutes les classes dans la hiérarchie d'héritage. C'est la stratégie de mappage d'héritage par défaut dans Entity Framework.
- Cela rend l'accès aux données très simple, mais repose sur l'utilisation de NULLs dans de nombreuses colonnes puisque la table finit par avoir l'ensemble complet de toutes les propriétés pour toutes les classes impliquées.



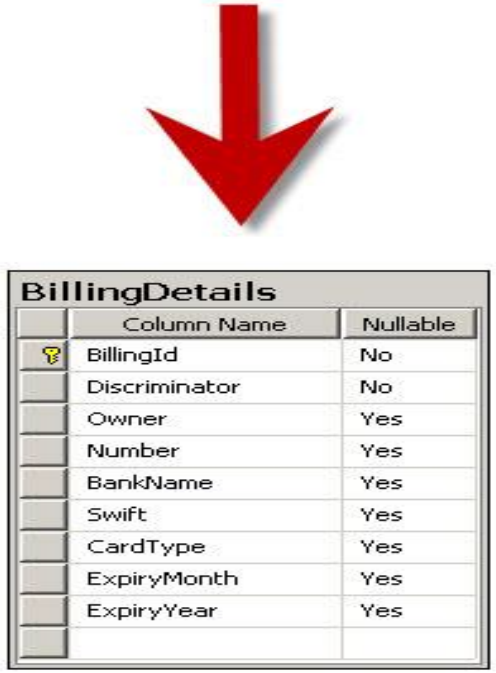


TABLE PER HIERARCHY (TPH)

Discriminator Column

- Comme vous pouvez le voir dans le schéma de base de données ci-dessus, Code First doit ajouter une colonne spéciale pour distinguer les classes persistantes .

- Le discriminateur. Ce n'est pas une propriété de la classe persistante dans notre modèle objet ; elle est utilisée en interne par EF Code First.

- Par défaut, le nom de la colonne est "Discriminator", et son type est string.

- Les valeurs par défaut sont les noms de classe persistants - dans ce cas, "BankAccount" ou "CreditCard".

TABLE PER HIERARCHY (TPH)

Dans notre modèle de domaine, nous avons une classe de base BillingDetail qui est abstraite. Nous avons deux sous-classes qui héritent de la classe BillingDetail.

```
public abstract class BillingDetail
{
    public int BillingDetailId { get; set; }
    public string Owner { get; set; }
    public string Number { get; set; }
}
```

```
public class BankAccount : BillingDetail
{
    public string BankName { get; set; }
    public string Swift { get; set; }
}
```

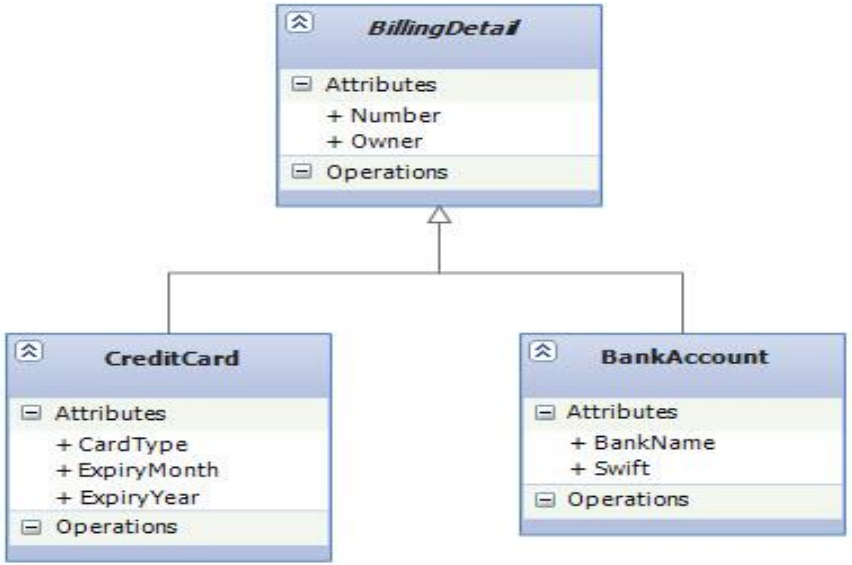
```
public class CreditCard : BillingDetail
{
    public int CardType { get; set; }
    public string ExpiryMonth { get; set; }
    public string ExpiryYear { get; set; }
}
```

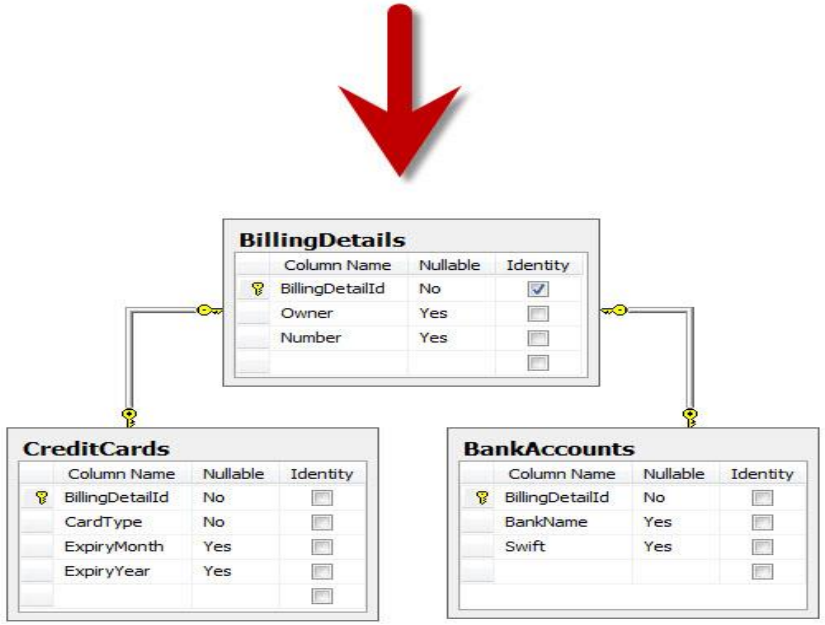
- Dans notre DbContext, je définis qu'un seul DbSet pour la classe de base qui est BillingDetail

```
public class InheritanceMappingContext : DbContext
{
    public DbSet<BillingDetail> BillingDetails { get; set; }
    public DbSet< BankAccount > BankAccount { get; set; }
    public DbSet< CreditCard > CreditCard { get; set; }
}
```

TABLE PER TYPE (TPT)

- Ce modèle utilise une table différente pour chaque classe (y compris les classes de base abstraites) dans la hiérarchie. Les propriétés héritées se trouvent dans la table "classe de base", et les propriétés des sous-classes sont dans leurs propres tables.
- Ces tables "dérivées" ont des colonnes de clés primaires ID qui sont une clé étrangère à la clé primaire de la table de la classe de base...





IMPLEMENT TPT IN EF CODE FIRST

Nous pouvons créer un mappage TPT simplement en plaçant un attribut Table sur les sous-classes pour spécifier le nom de la table mappée (l'attribut Table est une nouvelle annotation de données et a été ajouté à System.ComponentModel.DataAnnotations namespace

```
[Table("BillingDetail")]
public abstract class BillingDetail
{
    public int BillingDetailId { get; set; }
    public string Owner { get; set; }
    public string Number { get; set; }
}
[Table("BankAccounts")]
public class BankAccount : BillingDetail
{
    public string BankName { get; set; }
    public string Swift { get; set; }
}
```

```
[Table("CreditCards")]
public class CreditCard : BillingDetail
{
    public int CardType { get; set; }
    public string ExpiryMonth { get; set; }
    public string ExpiryYear { get; set; }
}
```

- Dans notre DbContext, je définis les trois DbSet pour les trois classes.

```
public class InheritanceMappingContext : DbContext
{
    public DbSet<BillingDetail> BillingDetails { get; set; }
    public DbSet<CreditCard> CreditCards { get; set; }
    public DbSet<BankAccount> BankAccounts { get; set; }
}
```